

Relatório de Requisitos do Sistema

Data de referência: 26/05/2026

Sistema: ObservaATT PE (registro e monitoramento de acidentes de trânsito em Pernambuco)

1. Escopo

Este documento consolida os requisitos do sistema conforme implementação atual do projeto, cobrindo:

- Requisitos funcionais
- Requisitos não funcionais
- Requisitos de segurança e privacidade
- Requisitos de dados
- Requisitos de integração e API
- Requisitos de infraestrutura e implantação
- Requisitos de operação e continuidade

2. Arquitetura e componentes

- Frontend web em HTML, CSS e JavaScript (PWA)
- Backend em Flask (Python)
- Persistência principal em Supabase (quando configurado)
- Persistência de fallback/local em arquivos JSON
- Exportações em CSV e mapa diário em HTML
- Empacotamento iOS com Capacitor

3. Requisitos Funcionais (RF)

3.1 Registro e gestão de ocorrências

- RF-001: O sistema deve permitir cadastro de acidentes de trânsito via API.
- RF-002: O sistema deve aceitar dados de localização geográfica do evento (latitude/longitude), com prioridade para marcação em tempo real.
- RF-003: O sistema deve aceitar dados do notificante (ex.: nome) e dados descritivos da ocorrência.
- RF-004: O sistema deve aceitar upload de evidências fotográficas da ocorrência.
- RF-005: O sistema deve permitir consulta da lista de ocorrências cadastradas.
- RF-006: O sistema deve fornecer metadados de ocorrências (total, última ocorrência e fingerprint de consistência).
- RF-007: O sistema deve permitir remoção de ocorrência por identificador.

3.2 Distribuição e visualização de dados

- RF-008: O sistema deve disponibilizar painel web principal para visualização analítica dos dados.
- RF-009: O painel deve apresentar ranking institucional de registros por órgão notificante.
- RF-010: O painel deve apresentar indicadores estatísticos operacionais e resumo executivo.
- RF-011: O painel deve incluir componente cartográfico de Pernambuco.

3.3 Exportação e monitoramento

- RF-012: O sistema deve gerar exportações por período (diário, semanal e mensal) em CSV.
- RF-013: O sistema deve disponibilizar download das exportações por endpoint administrativo.
- RF-014: O sistema deve disponibilizar download de mapa diário em HTML.
- RF-015: O sistema deve prover stream de atualizações em tempo real via SSE para clientes conectados.

3.4 Administração

- RF-016: O sistema deve autenticar acesso administrativo por chave de administrador.
- RF-017: O sistema deve permitir disparo manual de backup administrativo.
- RF-018: O sistema deve disponibilizar endpoint de diagnóstico de status do Supabase.
- RF-019: O sistema deve permitir sincronização administrativa para armazenamento em nuvem (quando habilitada).
- RF-020: O sistema deve expor endpoint para acesso ao link de planilhas/tabela configurado no Supabase.

4. Requisitos Não Funcionais (RNF)

4.1 Disponibilidade e confiabilidade

- RNF-001: O sistema deve manter modo de fallback local quando houver indisponibilidade de Supabase.
- RNF-002: O endpoint de saúde deve sinalizar degradação quando requisitos de persistência obrigatória não forem atendidos.
- RNF-003: O sistema deve operar com persistência em disco em ambiente de produção quando exigido por configuração.

4.2 Desempenho e limites

- RNF-004: O backend deve limitar tamanho de payload HTTP para upload (8 MB).

- RNF-005: O sistema deve limitar quantidade de fotos por registro (5 imagens).
- RNF-006: O sistema deve limitar tamanho de conteúdo por foto (controle de caracteres para payload codificado).
- RNF-007: Assets estáticos não HTML devem ser cacheáveis para ganho de desempenho.

4.3 Compatibilidade

- RNF-008: O frontend deve ser compatível com navegadores modernos com suporte à geolocalização.
- RNF-009: O sistema deve permitir instalação como PWA.
- RNF-010: O projeto deve suportar empacotamento iOS via Capacitor com backend HTTPS.

4.4 Manutenibilidade e observabilidade

- RNF-011: O sistema deve disponibilizar endpoints de diagnóstico e saúde para operação.
- RNF-012: O sistema deve manter estrutura de configuração por variáveis de ambiente.
- RNF-013: O sistema deve manter controle de versão da aplicação via endpoint dedicado.

5. Requisitos de Segurança e Privacidade (RSI)

- RSI-001: Endpoints administrativos devem exigir autenticação por chave administrativa.
- RSI-002: Credenciais de serviço Supabase não devem ser expostas no frontend.
- RSI-003: O sistema deve suportar criptografia de dados em repouso quando chave de criptografia for configurada.
- RSI-004: O sistema deve retornar cabeçalhos de não-cache para recursos sensíveis/dinâmicos.
- RSI-005: O sistema deve registrar e tratar erros de parsing e payload inválido sem interrupção do serviço.
- RSI-006: O sistema deve restringir operações administrativas ao escopo autenticado.

6. Requisitos de Dados (RD)

- RD-001: O sistema deve armazenar ocorrências em estrutura padronizada e normalizada.
- RD-002: O sistema deve preservar dados mínimos de identificação da notificação (id, data/hora, origem notificante).
- RD-003: O sistema deve suportar dados geográficos da ocorrência (coordenadas).

- RD-004: O sistema deve suportar dados de evidência (fotos) com limites definidos.
- RD-005: O sistema deve manter arquivo local de espelho e backup de segurança quando aplicável.
- RD-006: O sistema deve suportar sincronização de dados com tabela Supabase configurável.

7. Requisitos de API e Integração (RAI)

Endpoints principais implementados:

- GET /
- GET /health
- GET /version
- GET /
- GET /api/accidents
- POST /api/accidents
- DELETE /api/accidents/
- GET /api/accidents/meta
- GET /api/accidents/stream
- GET /api/exports
- GET /api/exports/download/
- GET /api/exports/download/daily-map
- POST /api/admin/auth
- POST /api/admin/backup-now
- GET /api/admin/spreadsheets-link
- GET /api/admin/supabase-status
- POST /api/admin/cloud-storage-sync

8. Requisitos de Infraestrutura e Implantação (RINF)

8.1 Plataforma e runtime

- RINF-001: Runtime Python para backend (configurado para Python 3.11 em implantação Render).
- RINF-002: Servidor WSGI Gunicorn para produção.
- RINF-003: Dependências geoespaciais e analíticas instaladas conforme requirements.

8.2 Variáveis de ambiente críticas

Obrigatórias em produção (conforme cenário):

- ADMIN_ACCESS_KEY
- DATA_DIR (recomendado /var/data em Render)
- REQUIRE_PERSISTENT_STORAGE

Obrigatórias para modo Supabase:

- SUPABASE_URL
- SUPABASE_SERVICE_ROLE_KEY
- SUPABASE_TABLE

Opcionais/operacionais:

- SUPABASE_BOOTSTRAP_LOCAL
- SUPABASE_STORAGE_SYNC_ENABLED
- SUPABASE_STORAGE_BUCKET
- SUPABASE_STORAGE_PREFIX
- SUPABASE_STORAGE_SYNC_INTERVAL_SECONDS
- SUPABASE_SPREADSHEETS_URL
- APP_TIMEZONE
- APP_VERSION

8.3 Empacotamento móvel

- RINF-004: iOS wrapper via Capacitor deve apontar para backend HTTPS configurado.
- RINF-005: Sincronização de assets web para pasta ios_web deve ocorrer antes do sync iOS.

9. Requisitos de Operação e Continuidade (ROC)

- ROC-001: O sistema deve permitir operação com fallback local em falha de banco remoto.
- ROC-002: O sistema deve suportar backups administrativos para repositório remoto quando configurado.
- ROC-003: O sistema deve gerar exportações periódicas para auditoria e compartilhamento.
- ROC-004: O endpoint /health deve ser utilizado como verificação obrigatória pós-deploy.
- ROC-005: O sistema deve manter pasta de exportações operacional com artefatos gerados.

10. Dependências técnicas mínimas

Bibliotecas críticas atualmente utilizadas:

- Backend/API: Flask, Werkzeug, Gunicorn, python-dotenv
- Segurança: cryptography
- Banco/integração: supabase
- Processamento e dados: numpy, pandas
- Geoespacial/cartográfico: geopandas, shapely, pyproj, fiona, rasterio, contextily, xyzservices, geobr
- Utilitários visuais: matplotlib, scipy, Pillow, qrcode
- Mobile wrapper: @capacitor/core, @capacitor/cli

11. Critérios de aceite recomendados

- CA-001: Cadastro, listagem e exclusão de ocorrência funcionando via API.
- CA-002: Fluxo com Supabase ativo e saudável validado no endpoint /health.
- CA-003: Fluxo degradado controlado com fallback local em indisponibilidade de Supabase.
- CA-004: Exportações diária/semanal/mensal geradas e disponíveis para download.
- CA-005: Autenticação administrativa por chave validada.
- CA-006: Painel web exibindo ranking, indicadores e componente cartográfico.
- CA-007: Build de produção executando com Gunicorn e variáveis de ambiente configuradas.

12. Observações finais

- Este relatório representa os requisitos consolidados do estado atual do sistema no repositório.
- Para governança formal, recomenda-se versionar este documento junto de matriz de rastreabilidade (requisito -> endpoint -> teste).
- Recomenda-se também anexar plano de testes funcionais e de segurança para homologação institucional.